学号：202310310083




# 《算法设计与分析在港航物流中的实践》课程设计（报告）

| | |
|---|---|
| 题　　目： | 港口集卡路径规划与任务分配 |
| 班　　级： | 计算机 231 |
| 姓　　名： | 倪嘉宏 |
| 同组人员： | 潘飞、冯方舟 |
| 指导教师： | 肖燕 |
| 完成日期： | 2026 年 6 月 |

上海海事大学信息工程学院

2026 年 6 月

# 目录

摘要.....1
1、引言.....2
1.1 动机.....2
1.2 要解决的问题.....2
1.3 本文的工作.....2
2、相关工作.....3
3、数据收集与标注.....4
3.1 港口环境数据.....4
3.2 任务数据.....4
3.3 拥堵标注数据.....5
4、关键方法.....5
4.1 地图初始化.....5
4.2 任务系统设计.....7
4.2.1 任务优先级计算设计.....7
4.2.2 任务生成器设计.....8
4.2.3 任务类型生成机制.....8
4.2.4 任务管理器设计.....8
4.2.5 任务分配机制.....9
4.2.6 任务完成处理.....9
4.2.7 调度性能统计.....10
4.3 机器学习任务价值预测.....10
4.3.1 特征提取机制设计.....10
4.3.2 随机森林集成预测机制.....11
4.3.3 队列长度自适应调节.....11
4.3.4 在线学习机制设计.....12
4.3.5 模型自适应更新机制.....12
4.4 基于匈牙利算法的任务分配策略设计.....12
4.4.1 任务分配模型构建.....13
4.4.2 最大距离归一化设计.....13
4.4.3 基于机器学习的任务价值评估.....13
4.4.4 最大收益转最小代价机制.....13
4.4.5 分配结果生成.....14

4.5 增强型 A*路径规划.....14
4.5.1 路径搜索.....15
4.5.2 实时拥堵避让机制设计.....15
4.5.3 拥堵预测机制设计.....15
4.5.4 综合代价函数设计.....16
4.5.5 路径更新机制.....16
4.5.6 路径搜索缓存优化设计.....16
4.6 拥堵趋势预测.....17
4.6.1 拥堵状态更新机制.....17
4.6.2 拥堵热点识别机制.....17
4.6.3 拥堵趋势预测机制.....17
4.7 多智能体强化学习.....18
4.7.1 动作空间设计.....18
4.7.2 状态空间设计.....18
4.7.3 Q 表设计.....19
4.7.4 ε-Greedy 决策机制.....19
4.7.5 Q-Learning 学习机制.....19
4.7.6 探索率衰减机制.....20
4.7.8 任务冲突消解机制.....20
4.7.9 奖励函数设计.....20
4.8 仿真控制器.....21
5、实验.....22
5.1 实验数据.....22
5.2 对比算法.....22
5.3 评估方法.....22
5.4 实验配置.....22
5.5 实验结果.....23
6、分析与讨论.....26
7、交互界面.....27
8、总结.....28
8.1 完成内容.....28
8.2 分工说明.....28
8.3 心得体会.....28
8.4 未来改进方向.....29
参考文献.....30

# 摘要

随着港口吞吐量的持续增长，集卡运输作为港口物流系统中的关键环节，其调度效率直接影响港口整体运营水平。针对传统调度方式存在任务分配效率低、路径规划缺乏拥堵感知以及多车协同能力不足等问题，本文设计并实现了一套港口集卡路径规划与任务分配智能调度系统。系统构建了可配置规模的港口仿真环境，在任务分配方面，采用融合随机森林预测模型的匈牙利算法，实现任务价值评估与全局最优匹配；在路径规划方面，提出基于增强型 A*算法的动态路径规划方法，引入实时拥堵感知和历史趋势预测机制，提高路径选择的合理性；同时结合多智能体 Q-Learning 强化学习框架，实现车辆自主决策与协同调度。系统基于 Flask 与 Canvas 开发可视化平台，支持实时监控与策略对比实验。实验结果表明，与传统基线方法相比，所提出系统能够有效降低任务平均等待时间，提高运输效率和资源利用率，在不同规模港口场景下均表现出良好的稳定性和扩展性，验证了方案的可行性与有效性。

**关键词：**港口物流；集卡调度；路径规划；A*算法；匈牙利算法；强化学习；拥堵预测；任务分配。

# 1、引言

随着全球贸易规模的不断扩大，港口在国际物流体系中的枢纽作用愈发重要，其中集卡运输承担着集装箱在码头、堆场与闸口之间的核心转运任务。由于港口内部路网结构复杂、作业任务密集且具有动态性，传统依赖经验的调度方式难以实现高效运行，容易造成运输路径冗余和资源利用率低等问题。因此，将算法设计与分析方法引入港口物流系统，对集卡路径规划与任务调度进行优化，不仅能够提高运输效率、降低运营成本，还能为智慧港口建设提供理论支持和技术基础，具有较强的现实意义和工程应用价值。

## 1.1 动机

全球贸易的持续增长使得港口吞吐量逐年攀升。在自动化集装箱码头中，水平运输环节（即集装箱卡车在泊位与堆场之间的往返运输）是整个作业链条的关键瓶颈。据统计，集卡在港内的无效等待时间占整体作业时间的 30%~40%，这直接影响了港口的整体作业效率和运营成本。

港口集卡调度问题的核心难点在于：（1）动态性——任务随机到达，车辆状态实时变化；（2）耦合性——多车路径相互影响，局部拥堵可能引发连锁延误；（3）多目标性——需同时优化任务响应时间、车辆利用率和路径拥堵程度。传统的规则式调度策略难以有效应对上述复杂性，亟需引入智能化方法。

## 1.2 要解决的问题

本课程设计围绕以下四个核心问题展开：

（1）任务分配优化：从全局最优视角，将动态到达的等待任务分配给最合适的空闲车辆，使得总体运输成本最低且任务响应最快

（2）路径规划优化：在多车并发的动态环境中为每辆车规划路径，不仅避开当前拥堵，还能提前规避历史高频拥堵区域

（3）多车协同优化：让多辆车在竞争有限道路资源的条件下实现合作，减少基于硬编码 ID 优先级的避让规则，提高系统的自主性和灵活性

（4）系统可视化：直观展示调度系统的实时运行状态、拥堵分布和性能指标，并支持不同策略配置的便捷对比

## 1.3 本文的工作

本文的主要贡献如下：

（1）设计了可配置规模的随机港口布局生成器（8×3 至 20×10），包含泊位区、

堆场块和网格化巷道网络，支持动态间距压缩以适应不同规模场景。

（2）构建了增强型 A*路径规划器，在标准启发式搜索基础上集成"即时拥堵感知"与"历史趋势预测"双层惩罚机制，实现了从被动避障到主动绕行的演进。

（3）提出了 ML 增强的匈牙利任务分配算法，以随机森林回归融合四维特征（紧急度、距离、等待时间、任务类型）构建任务价值函数，替代纯距离静态成本模型。

（4）开发了 Q-Learning 多智能体调度框架，为每辆车配备独立 RL-Agent，支持 epsilon-greedy 探索、Q 值比较冲突消解和在线策略学习。

（5）搭建了基于 Flask+Canvas 的 Web 可视化平台，实现仿真计算与前端渲染的完全分离，支持交互式参数调节和实时性能监控。

# 2、相关工作

港口物流调度问题是运筹学与人工智能领域的经典研究课题，涉及任务分配、路径规划和多智能体协同三个核心子问题。

在任务分配方面，匈牙利算法（Kuhn, 1955）是解决二分图最小权匹配的最优方法，时间复杂度为 $O(n^3)$，被广泛应用于车辆-任务匹配场景[1]。近年来，部分研究将机器学习引入分配决策，如使用梯度提升树预测任务的紧急度，或利用深度神经网络学习启发式分配策略。

在路径规划方面，A*算法（Hart et al., 1968）作为经典的启发式搜索方法，在已知静态路网中能保证找到最优路径[2]。针对动态环境，D* Lite[3]和 Anytime Repairing A*[4]等增量式重规划算法被提出。在港口场景下，车辆行驶受限于巷道网络，拥堵多发生在泊位入口和堆场内部路口——这些位置也是本文设计的拥堵预测器重点监控的区域。

在多智能体协同方面，基于规则的方法（如优先级避让、分段锁）实现简单但灵活性不足。基于强化学习的方法，如 MADDPG[5]和 QMIX[6]，通过集中训练-分散执行的范式实现智能体间的隐式协作，但训练成本较高，且对仿真环境依赖性强。

与上述工作相比，本文的特点在于：将任务价值预测、拥堵趋势学习和多智能体 Q-Learning 有机集成于统一的仿真框架中，各模块可独立开关以进行消融实验；系统通过 Web 界面提供直观的交互式分析工具，可视化展示调度策略。

# 3、数据收集与标注

## 3.1 港口环境数据

本系统的港口环境通过随机生成器自动构建，无需外部数据集。港口布局由以下参数控制：

**表 1. 港口生成参数**

| 参数 | 说明 | 取值范围 | 默认值 |
|---|---|---|---|
| num_cols | 港口网格列数 | 8-20 | 8 |
| num_rows | 堆场行数 | 3-10 | 3 |
| yard_prob | 堆场块生成概率 | 0-1 | 0.7 |
| quay_prob | 泊位生成概率 | 0-1 | 0.6 |

每次运行使用固定随机种子（seed=42），保证实验可复现。生成的港口环境包含四类节点：泊位节点 Q、巷道节点（C_L/C_R，左右各一）、主干道节点（每列底部 L0/R0）、出入口节点（Gate_In/Gate_Out，位于港口两端）。路网以无向图邻接表存储，双向连接形成完全连通的网格拓扑。

## 3.2 任务数据

任务也是随机生成，包含三种类型，各自对应不同的典型港口作业场景：

**表 2. 任务类型与特征**

| 类型 | 描述 | 起点 | 终点 | 生成权重 | 紧急度特征 |
|---|---|---|---|---|---|
| IMPORT | 进口卸船 | 泊位(Quay) | 堆场(Yard) | 40% | 偏高 (0.5~1.0) |
| EXPORT | 出口装船 | 闸口 (Gate_In) | 堆场(Yard) | 30% | 中等 (0.3~0.8) |
| TRANSSHIPMENT | 堆场转运 | 堆场(Yard) | 泊位(Quay) | 30% | 中等 (0.3~0.8) |

每个任务携带以下标注信息：urgency（紧急度，0~1 浮点数）、deadline（截止时间戳）、waiting_since（任务创建/等待起始时刻）、priority_score（综合优先级评分）。IMPORT 类型任务的紧急度默认加上 0.2 的偏置，以反映实际港口作业中进口卸船任务的时间紧迫性。

### 3.3 拥堵标注数据

仿真运行过程中，CongestionPredictor 每步自动采集并标注拥堵数据。空间标注将港口区域离散化为 10×10 单位的矩形网格，统计每格内的车辆分布。时间标注为每个网格保留最近 50 步的密度历史序列。EWMA 值≥0.6 的网格自动标记为拥堵热点。

## 4、关键方法

### 4.1 地图初始化

港口布局采用网格化随机生成方法。以入口（Gate_In）为起点、出口（Gate_Out）为终点，按列（下记 c，从 1 到 num_cols）和行（下记 r，从 1 到 num_rows）组织空间结构。每列包含一个泊位节点 Q{c}、两个主干道节点 C{c}_L0 和 C{c}_R0，以及每行两个堆场节点 C{c}_L{r}和 C{c}_R{r}。

节点连接采用"纵向巷道+横向主干道"的网格拓扑：L 侧节点纵向串联形成左巷道，R 侧节点纵向串联形成右巷道；每行的 L 和 R 节点间若存在堆场块则横向连接；每列的 L0 和 R0 通过泊位节点 Q{c}形成三角形连接；相邻列之间通过横向主干道（C{c-1}_R0 → C{c}_L0）连通。

当用户参数为大规模场景时自动启用动态间距压缩，保证可视化效果。

动态间距实现代码：

```
if num_cols > 12:
                self.col_spacing = 14
                self.row_spacing = 10
        elif num_cols > 16:
                self.col_spacing = 12
                self.row_spacing = 9
        else:
                self.col_spacing = 18
                self.row_spacing = 12
```

生成泊位：

```
x_left = c * self.col_spacing - 5
x_right = c * self.col_spacing + 5
q_node = f'Q{c}'
self._add_node(q_node, ((x_left + x_right) / 2, 2))
if random.random() < quay_prob:
    self.zones.append({
        'pos': (x_left - 2, 0.5),
        'w': (x_right - x_left) + 4,
        'h': 3, 'color': '#708090',
        'label': f'Quay {c}'
    })
```

生成主干道：

```
n_in = f'C{c}_L0'
n_out = f'C{c}_R0'
self._add_node(n_in, (x_left, self.base_y))
self._add_node(n_out, (x_right, self.base_y))
self._add_edge(n_in, n_out, bidirectional=True)
self._add_edge(n_in, q_node, bidirectional=True)
self._add_edge(q_node, n_out, bidirectional=True)
column_path_left = [n_in]
column_path_right = [n_out]
```

生成堆场：

```
y_pos = self.base_y + r * self.row_spacing
node_l = f'C{c}_L{r}'
node_r = f'C{c}_R{r}'
self._add_node(node_l, (x_left, y_pos))
self._add_node(node_r, (x_right, y_pos))
if random.random() < yard_prob:
    self.yards.append({
        'pos': (x_left + 2, y_pos - 4),
        'w': (x_right - x_left) - 4,
        'h': 8,
        'label': f'Block {c}-{r}'
    })
    self._add_edge(node_l, node_r, bidirectional=True)
self._add_edge(column_path_left[-1], node_l, bidirectional=True)
self._add_edge(column_path_right[-1], node_r, bidirectional=True)
column_path_left.append(node_l)
column_path_right.append(node_r)
```

在横向结构上，系统通过跨列连接机制保证港口整体连通性，即当前列入口与前一列出口之间建立边连接，从而形成连续的运输通道结构，确保车辆可以在整个港口范围内自由通行。

```
self._add_edge(f'C{c-1}_R0', n_in, bidirectional=True)
```

上列所述的_add_edge 定义如下：

```
def _add_edge(self, u, v, bidirectional=False):
    self.graph[u].append(v)
    if bidirectional:
        self.graph[v].append(u)
```

## 4.2 任务系统设计

为了实现港口运输任务的统一管理，系统设计了 Task 类用于描述单个运输任务。该类封装了任务编号、任务类型、起始节点、目标节点、任务状态、紧急程度以及截止时间等信息。同时，为支持后续调度决策，任务对象还记录等待时间、优先级得分和预估运输距离等动态属性。通过面向对象设计，可以将任务相关的数据和操作集中管理，提高系统的可维护性和扩展性。

```
class Task:
    def __init__(self, task_id, task_type, start_node, end_node,urgency=None,deadline=None):
        self.id = task_id
        self.type = task_type
        self.start = start_node
        self.end = end_node
        self.status = "PENDING"
        self.urgency = urgency if urgency is not None else random.uniform(0.3, 1.0)
        self.deadline = deadline if deadline is not None else time.time() + random.uniform(60,
300)
        self.created_at = time.time()
        self.waiting_since = time.time()
        self.priority_score = 0.0
        self.estimated_distance = 0.0
```

### 4.2.1 任务优先级计算设计

在调度过程中，不同任务的重要程度存在明显差异。为了兼顾运输效率和系统公平性，系统采用综合优先级评价机制，从任务紧急程度、运输距离以及等待时间三个维度对任务进行评分。其中，紧急程度反映任务的重要性，距离因素体

现运输成本，而等待时间则用于避免部分任务长期得不到执行。最终采用加权求和的方法得到任务优先级，为调度器提供决策依据。

```
def compute_priority(self, distance_cost, max_distance, waiting_penalty_weight=0.3):
        distance_norm = distance_cost / max(max_distance, 1.0)
        waiting = time.time() - self.waiting_since
        waiting_norm = min(waiting / 120.0, 1.0)
        self.priority_score = (0.4 * self.urgency +0.3 * (1.0 - distance_norm) +waiting_penalty_w
eight * waiting_norm)
        self.estimated_distance = distance_cost
        return self.priority  score
```

### 4.2.2 任务生成器设计

实际港口作业中，运输任务具有随机到达特性。为了模拟真实场景并为调度算法提供测试数据，系统设计了任务生成器。该模块能够根据港口拓扑结构自动识别岸桥节点和堆场节点，并按照预设概率随机生成进口、出口和中转三种类型任务，从而构建具有随机性的任务流。

```
self.quay_nodes = [n for n in port.nodesif n.startswith('Q')]
self.yard_nodes = [n for n in port.nodesif ('_L' in n or '_R' in n)and not n.endswith('0')]
```

### 4.2.3 任务类型生成机制

为了符合实际港口业务特点，系统按照一定概率生成不同类型任务。其中进口任务占 40%，出口任务占 30%，中转任务占 30%。由于进口任务通常具有更高时效性要求，因此在生成时会额外增加紧急度参数，使其在后续调度中获得更高优先级。

```
types = ['IMPORT', 'EXPORT', 'TRANSSHIPMENT']
type_weights = [0.4, 0.3, 0.3]
t_type = random.choices(types,weights=type_weights,k=1)[0]
urgency_bonus = 0.2 if t_type == 'IMPORT' else 0.0
urgency = min(1.0,random.uniform(0.3, 0.8) + urgency_bonus)
```

### 4.2.4 任务管理器设计

为了统一管理系统中的所有运输任务，设计了 TaskManager 类。该模块负责维护任务队列、记录已完成任务、保存历史运行数据以及统计系统运行指标。通过集中式管理，可以实现任务状态跟踪和调度结果分析，为后续优化算法提供数

据支持。

```
class TaskManager:
    def __init__(self, tasks):
        self.queue = tasks
        self.completed_tasks = []
        self.task_history = []
        self.total_completed = 0
        self.total_waiting_time = 0.0
```

### 4.2.5 任务分配机制

当调度器空闲时，调度器需要从任务队列中选取待执行任务。系统通过遍历任务队列寻找状态为 PENDING 的任务，并将其状态更新为 RUNNING 后分配给车辆执行。

```
def get_next_task(self):
    for task in self.queue:
        if task.status == "PENDING":
            task.status = "RUNNING"
            return task
    return None
```

### 4.2.6 任务完成处理

当完成运输任务后，系统需要更新任务状态并记录相关统计信息。TaskManager 通过 mark_done 函数完成任务状态更新、等待时间统计以及历史数据保存，为后续性能分析提供依据。

```
def mark_done(self, task):
    task.status = "COMPLETED"
    self.completed_tasks.append(task)
    self.total_completed += 1
    wait_time = time.time() - task.waiting_since
    self.total_waiting_time += wait_time
    self.task_history.append({
        "id": task.id,
        "type": task.type,
        "urgency": task.urgency,
        "wait_time": wait_time,
        "distance": task.estimated_distance
    })
```

### 4.2.7 调度性能统计

为了评估调度算法运行效果，系统设计了统计分析功能。该模块能够实时统计已完成任务数、剩余任务数、平均等待时间以及平均紧急度等指标，从而反映系统整体运行效率和任务服务水平。

```
def get_stats(self):
    avg_wait = (self.total_waiting_time / max(self.total_completed, 1))
    return {
        "completed": self.total_completed,
        "pending": len(self.get_pending_tasks()),
        "avg_wait_time": round(avg_wait, 2),
        "avg_urgency": round(sum(t.urgency for t in self.completed_tasks[-20:])
            / max(len(self.completed_tasks[-20:]), 1), 3
        ) if self.completed_tasks else 0
    }
```

## 4.3 机器学习任务价值预测

为提高任务分配的智能化水平，系统设计了任务价值预测模块。该模块借鉴随机森林回归思想，综合考虑任务紧急度、运输距离、等待时间以及任务类型等特征，对任务价值进行预测评分。评分结果用于辅助调度器选择最值得优先执行的任务，从而提高系统整体运行效率。

```
class TaskValuePredictor:
    def __init__(self, num_trees=10, max_depth=5):
        self.num_trees = num_trees
        self.max_depth = max_depth
        self.training_buffer = deque(maxlen=500)
       self.weights = {"urgency": 0.40, "distance": 0.25, "waiting": 0.20, "type_bonus": 0.15}
```

### 4.3.1 特征提取机制设计

为了准确描述任务状态，系统从任务对象和运行环境中提取多个关键特征构建特征向量。提取的特征包括任务紧急程度、车辆与任务之间的距离、任务等待时间以及任务类型编码。

```
def _extract_features(self, task, truck_pos, task_pos, context):
    return [urgency, distance_norm, waiting_norm, type_encoding]
```

任务类型编码设计：

```
type_map = {"IMPORT": 0.8,"EXPORT": 0.5,"TRANSSHIPMENT": 0.65}
type_encoding = type_map.get(task.type,0.5)
```

基础任务价值评估：不同特征根据其重要程度分配不同权重，其中紧急程度占比最高，其次为距离因素、等待时间和任务类型。

```
urgency_score = (features[0] * self.weights["urgency"])
distance_score = ((1.0 - features[1]) * self.weights["distance"])
waiting_score = (features[2] * self.weights["waiting"])
type_score = (features[3] * self.weights["type_bonus"])
```

### 4.3.2 随机森林集成预测机制

为了进一步提高预测能力，系统引入轻量级随机森林结构。模型由多个简化决策树组成，每棵树基于不同特征和分裂阈值进行评价，最终通过集成方式得到综合预测结果。相比单一规则模型，随机森林能够更好地刻画特征之间的非线性关系，提高任务价值评估的鲁棒性。

```
for tree in self.trees:
    f_idx = tree["split_feature"]
    if features[f_idx] > tree["split_value"]:
        adj = tree["left_weight"]
    else:
        adj = tree["right_weight"]
    tree_adjustments.append(adj)
avg_adjustment = (sum(tree_adjustments) / len(tree_adjustments) )
```

系统将基础评分与随机森林预测结果进行融合，形成最终任务价值评分。通过这种方式，既保留人工经验规则的可解释性，又引入机器学习模型的自适应能力。

```
base_score = (base_score * 0.9 + avg_adjustment * 0.1)
return min(1.0, max(0.0, base_score))
```

### 4.3.3 队列长度自适应调节

在高负载场景下，任务队列长度会显著增加。为了避免任务积压，系统根据当前队列规模动态调整任务价值评分。当任务数量较多时，提高评分增益，加快任务处理速度。该机制增强了调度系统对高峰期作业压力的适应能力。

```
queue_length = context.get("queue_length",0)
if queue_length > 5:
    base_score *= (1.0 +0.02 *min(queue_length - 5, 10))
```

### 4.3.4 在线学习机制设计

为了使模型能够持续适应港口运行环境变化，系统设计了在线学习机制。每当任务完成后，系统记录预测结果与实际执行效果，并保存至训练缓冲区。这些历史数据将作为模型后续优化的重要依据。

```
self.training_buffer.append({"features": [...],"predicted": predicted_score, "actual_utility":
        1.0 / max(actual_wait_time, 1.0) if was_completed else 0.0
})
```

### 4.3.5 模型自适应更新机制

当训练数据达到一定规模后，系统自动执行在线更新。通过分析预测误差大小，对随机森林中的分裂阈值进行微调，从而逐步提高模型预测精度。这种增量学习方式无需重新训练整个模型，适合实时运行环境。

```
if avg_error > 0.2:
    for tree in self.trees:
        tree["split_value"] += random.uniform(-0.05,0.05)
        tree["split_value"] = min(0.9,max(0.1, tree["split_value"]))
```

## 4.4 基于匈牙利算法的任务分配策略设计

传统调度方法通常采用最近距离优先或先来先服务策略，但这类方法仅关注局部最优，容易导致整体运输效率下降。因此，本系统采用匈牙利算法（Hungarian Algorithm）求解任务分配问题，将本问题建模为二分图最优匹配问题，通过构建代价矩阵寻找全局最优分配方案，从而提高资源利用率和运输效率。

任务分配前，系统首先统计当前所有空闲集卡和待执行任务。只有处于空闲状态的车辆才允许参与调度，而已完成或执行中的任务不会进入本轮分配。这种设计能够保证调度过程的实时性和正确性。

```
idle_trucks = [t for t in self.trucksif t.status == "IDLE"]
pending_tasks = [t for t in self.manager.queueif t.status == "PENDING"]
if len(idle_trucks) == 0 or len(pending_tasks) == 0:
    return {}
```

### 4.4.1 任务分配模型构建

为了使用匈牙利算法求解最优匹配问题，需要构建车辆与任务之间的代价矩阵。矩阵中：行表示集卡车辆；列表示待执行任务；元素表示某车辆执行某任务的代价。由于车辆数量与任务数量可能不一致，因此系统采用方阵扩展方式，将矩阵补齐至最大维度。

```
n_trucks = len(idle_trucks)
n_tasks = len(pending_tasks)
max_dim = max(n_trucks,n_tasks)
cost = np.full( (max_dim, max_dim),1e9)
```

### 4.4.2 最大距离归一化设计

由于任务分布位置不同，为保证机器学习模型输入特征的一致性，系统对距离特征进行归一化处理。归一化所需的最大距离由当前任务集合动态计算得到。

```
max_distance = max(
      self.dist( (0, 0), (self.port.nodes[t.start][0],self.port.nodes[t.start][1]))
      for t in pending_tasks
) if pending_tasks else 200.0
```

### 4.4.3 基于机器学习的任务价值评估

调用 4.3 中的机器学习代码对任务价值预测

```
context = {"max_distance":max_distance,"waiting_time":time.time() - task.waiting_since,
"queue_length":len(pending_tasks) }
score = self.task_predictor.predict_score(task,truck.current_pos, task_pos,context)
```

### 4.4.4 最大收益转最小代价机制

匈牙利算法本质上求解最小代价匹配问题，而任务价值预测模块输出的是收益评分。因此系统采用负值转换方式，将收益最大化问题转化为代价最小化问题，从而能够直接使用匈牙利算法求解。

```
cost[i][j] = -score * 100
```

目标函数为：

$$\min \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij} x_{ij}$$

约束条件为：

$$\sum_{j=1}^{n} x_{ij} \leq 1$$

$$\sum_{i=1}^{m} x_{ij} \leq 1$$

决策变量为：

$$x_{ij} \in \{0,1\}$$

调用 SciPy 的 linear_sum_assignment 求解器求解：

```
row_ind, col_ind = linear_sum_assignment(cost)
```

### 4.4.5 分配结果生成

根据匈牙利算法输出结果，系统建立车辆与任务之间的一一对应关系，并生成最终调度方案。

```
assignment = {}
for r, c in zip(row_ind, col_ind)
    if r < n_trucks and c < n_tasks:
        assignment[idle_trucks[r] ] = pending_tasks[c]
```

## 4.5 增强型 A*路径规划

标准 A*算法使用启发式函数 f(n) = g(n) + h(n) 指导搜索方向。本系统在 g(n)的计算中引入了创新性的双层拥堵惩罚机制：

**表 3. A*边代价构成**

| 代价层级 | 计算方式 | 权重 | 触发条件 | 来源 |
|---|---|---|---|---|
| 基础距离 | 欧几里得距离 dist(u, v) | 1.0 | 始终 | 节点坐标 |
| 即时惩罚 | Σ 40×(1−d/6)，∀卡车 d<6 | 可变 | 卡车在半径 6 内 | 当前车辆位置 |
| 趋势惩罚 | 15×max(当前拥堵，预测拥堵) | 15.0 | 拥堵值>0.1 | CongestionPredictor |

欧几里得距离：

```
def heuristic(pos1, pos2):
    return math.sqrt( (pos1[0] - pos2[0]) ** 2 + (pos1[1] - pos2[1]) ** 2)
```

### 4.5.1 路径搜索

系统采用优先队列维护开放节点集合，每次优先扩展总代价最小的节点。搜索过程中记录每个节点的最优父节点信息，当到达目标节点后，通过回溯父节点链即可恢复完整路径。该机制保证了路径搜索结果的最优性。

```
open_set = []
heapq.heappush(open_set, (0, start))
while open_set:
    current_f, current = heapq.heappop(open_set)
    if current == end:
        path = []
        while current in came_from:
            path.append(current)
            current = came_from[current]
        path.append(start)
        return path[::-1]
```

### 4.5.2 实时拥堵避让机制设计

传统 A*算法仅考虑道路长度，无法反映实际交通状况。为了提高调度运行效率，系统引入实时拥堵避让机制。当某节点附近存在其他运输车辆时，系统根据车辆距离动态增加路径代价。距离越近，拥堵惩罚越大，从而引导车辆主动绕开拥堵区域，减少等待时间和交通冲突。

```
instant_penalty = 0
if truck_positions:
    for t_pos in truck_positions:
        d_to_truck = heuristic(node_coords[neighbor], t_pos)
        if d_to_truck < 6.0:
            instant_penalty += (40.0 *(1.0 - d_to_truck / 6.0))
```

### 4.5.3 拥堵预测机制设计

仅依靠实时车辆位置无法提前规避潜在拥堵区域，因此系统进一步引入拥堵预测机制。该机制利用历史运行数据构建拥堵预测模型，对未来节点拥堵程度进行估计。当某节点被识别为高频拥堵区域时，其路径代价将被提前提高，使路径

规划具有前瞻性，从而减少车辆集中进入热点区域导致的交通瓶颈问题。

```
if congestion_predictor and enable_trend:
    base_cong = (congestion_predictor.get_node_congestion(neighbor))
    if congestion_predictor.is_hotspot(node_coords[neighbor][0], node_coords[neighbor][1]):
        base_cong = max(base_cong, 0.7)
    predicted = (congestion_predictor.predict_congestion(neighbor))
    trend_penalty = (congestion_weight *max(base_cong, predicted))
```

### 4.5.4 综合代价函数设计

为了同时考虑路径长度、实时拥堵以及未来拥堵趋势，系统构建综合代价函数作为 A*搜索依据。路径总代价由基础行驶距离、实时拥堵惩罚以及趋势拥堵惩罚三部分组成。通过多因素融合，路径规划结果不仅保证较短距离，还能够有效避开拥堵区域，提高整体运输效率。

```
tentative_g = (g_score[current]+ dist+ instant_penalty+ trend_penalty)
```

### 4.5.5 路径更新机制

当搜索发现更优路径时，系统更新当前节点的最优代价和父节点信息，并重新计算优先级后加入开放列表。该机制保证搜索过程始终保留当前最优解，从而最终获得总代价最小的运输路径。

```
if tentative_g < g_score[neighbor]:
    came_from[neighbor] = current
    g_score[neighbor] = tentative_g
    f_score = (tentative_g + heuristic(node_coords[neighbor], node_coords[end]))
    heapq.heappush(open_set, (f_score, neighbor) )
```

### 4.5.6 路径搜索缓存优化设计

在大规模港口场景下，调度器需要频繁进行路径规划，大量重复的距离计算会增加系统开销。为提高算法运行效率，系统利用 LRU（Least Recently Used）缓存机制对启发式距离计算结果进行存储。当相同节点对再次参与搜索时，可直接读取缓存结果，避免重复计算，从而提升整体规划速度。

```
@lru_cache(maxsize=512)
def _cached_heuristic(p1, p2):
    return math.sqrt( (p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)
```

## 4.6 拥堵趋势预测

在港口调度系统中，仅依赖即时车辆位置进行路径避障存在明显局限性：系统只能“被动避让当前拥堵”，但无法提前规避即将形成的交通热点区域。因此，本系统设计了拥堵预测模块，用于对港口道路空间中的交通密度进行建模与趋势预测。

### 4.6.1 拥堵状态更新机制

为了避免瞬时车辆分布变化导致拥堵估计剧烈波动，系统采用指数滑动平均（EWMA）对拥堵状态进行平滑更新。EWMA 能够在保留历史趋势的同时增强对近期变化的敏感性，从而更准确地反映真实交通状态。

```
self.heatmap[gkey] = (self.ewma_alpha * density + (1 - self.ewma_alpha) * old)
```

除了网格级别的拥堵统计，系统还针对关键节点（如岸桥、堆场交汇点）建立节点级拥堵模型。通过计算一定距离范围内的车辆数量，得到节点局部拥堵强度，并同样使用 EWMA 进行平滑更新。

```
for name, count in node_counts.items():
    density = min(count / 3.0, 1.0)
    old = self.node_congestion[name]
    self.node_congestion[name] = (self.ewma_alpha * density +(1 - self.ewma_alpha) * old)
```

### 4.6.2 拥堵热点识别机制

为了识别长期高负载区域，系统定期（每 20 步仿真）更新热点集合。当某个网格的拥堵均值超过设定阈值时，该区域被标记为热点。热点区域在路径规划中会被赋予更高代价，从而引导集卡主动绕行

```
def _update_hotspots(self):
    self.hotspots = set()
    for gkey, val in self.heatmap.items():
        if val >= self.hotspot_threshold:
            self.hotspots.add(gkey)
```

### 4.6.3 拥堵趋势预测机制

除了当前拥堵状态，系统进一步引入短期趋势预测模型。通过分析最近一段时间的密度变化，计算增长趋势并进行线性外推，从而预测未来短时间内的拥堵

情况。该机制使系统具备“提前规避潜在拥堵”的能力，而不仅仅依赖当前状态。

```
recent = history[-5:]
trend = (recent[-1] - recent[0]) / max(len(recent), 1)
predicted = current + trend * lookahead_steps
return max(0.0, min(1.0, predicted))
```

拥堵预测模块与 A*算法深度结合，在路径代价函数中引入以下信息，从而避免当前拥堵，还会主动绕开未来可能形成的高风险区域。

```
def get_node_congestion(self, node_name):
    return self.node_congestion.get(node_name, 0.0)
def is_hotspot(self, x, y):
    return self._to_grid(x, y) in self.hotspots
```

## 4.7 多智能体强化学习

传统调度算法通常依赖预定义规则进行任务分配，虽然能够获得较好的短期效果，但难以适应复杂动态环境。随着港口规模扩大和任务数量增加，调度集卡需要根据环境状态自主调整决策策略。因此，本系统引入强化学习机制，为每辆集卡配置独立的 Q-Learning 智能体。智能体通过与环境持续交互，根据任务完成情况、车辆空闲时间以及冲突风险等反馈不断优化决策策略，从而实现自主学习与智能调度。

### 4.7.1 动作空间设计

强化学习智能体需要在每个决策周期选择一个动作。结合港口集卡运行特点，系统定义四种典型决策行为。包括：接受最近任务、接受最高优先级任务、保持空闲等待、前往中心区域待命。 通过有限动作空间设计，可以降低学习复杂度，提高训练效率。

```
ACTIONS = {0: "accept_nearest",1: "accept_highest",2: "wait_idle",3: "relocate_center"}
```

### 4.7.2 状态空间设计

强化学习决策效果高度依赖环境状态描述。由于连续状态空间维度较高，系统采用离散化方法构建状态表示。

状态由以下四部分组成：当前是否已分配任务、周边车辆数量、最近待执行任务距离、当前任务队列长度。离散化后的状态能够有效降低 Q 表规模，提高学习效率。

```
state = (has_task, nearby_vehicle_bin,nearest_task_bin,queue_length_bin)
return f"{has_task}_{nb_bin}_{dist_bin}_{qlen_bin}"
```

为了将连续变量映射到有限状态空间，系统采用区间划分方式进行离散化处理。

```
def _discretize(self, value, bins, min_val, max_val):
    norm = ((clamped - min_val) / max(max_val - min_val, 1e-6) )
    return min(int(norm * bins), bins - 1)
```

### 4.7.3 Q 表设计

系统采用经典 Q-Learning 算法，不依赖深度神经网络，而是使用字典结构维护状态—动作价值表。Q 值表示在某状态下执行某动作所获得的长期累计收益。

```
self.q_table = defaultdict(lambda: [0.0, 0.0, 0.0, 0.0] )
```

### 4.7.4 ε-Greedy 决策机制

为了平衡探索与利用，系统采用 ε-Greedy 策略选择动作。即当随机数小于 ε 时随机探索，否则选择当前 Q 值最大的动作

```
if explore and random.random() < self.epsilon:
    action = random.randrange(len(self.ACTIONS) )
else:
    max_q = max(q_values)
    best_actions = [i for i, q in enumerate(q_values) if q == max_q]
    action = random.choice(best_actions)
```

### 4.7.5 Q-Learning 学习机制

在获得环境反馈后，系统采用 Q-Learning 公式更新状态动作价值。更新规则如下：

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max\ Q(s',a') - Q(s,a)]$$

其中：α 为学习率；γ 为折扣因子；r 为即时奖励；s' 为下一状态。

```
current_q = self.q_table[state_key][action]
max_next_q = max(self.q_table[next_state_key])
new_q = current_q + self.lr * (reward +self.gamma * max_next_q -current_q)
self.q_table[state_key][action] = new_q
```

### 4.7.6 探索率衰减机制

随着训练次数增加，智能体应逐渐减少随机探索行为，转而利用已经学习到的策略。因此系统采用指数衰减方式更新探索率。

```
def decay_epsilon(self):
    self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)
```

### 4.7.8 任务冲突消解机制

当多个集卡同时选择同一个任务时，系统采用竞争机制进行冲突消解。具体规则：比较对应动作的 Q 值，Q 值较高的车辆获得任务，其余车辆进入等待状态。这种设计能够减少重复抢占任务现象。

```
if best_task.id in task_assignments:
    other_truck, other_q = task_assignments[best_task.id]
    if q_value > other_q:
        task_assignments[best_task.id] = (truck_id, q_value)
        resolved[truck_id] = ("accept", best_task)
        resolved[other_truck] = ("wait", None)
        self.conflict_count += 1
    else:
        resolved[truck_id] = ("wait", None)
else:
    task_assignments[best_task.id] = (truck_id, q_value)
    resolved[truck_id] = ("accept", best_task)
```

### 4.7.9 奖励函数设计

奖励函数是强化学习系统的核心组成部分。设计如下完成任务+10、碰撞风险-5、空闲每步-0.05×duration、无任务移动-0.5。

```
reward = 0.0
if was_completed:
    reward += 10.0
if had_collision_risk:
    reward -= 5.0
if truck.status == "IDLE":
    reward -= 0.05 * min(idle_duration, 50)
if action_result == "relocate":
    reward -= 0.5
```

## 4.8 仿真控制器

仿真相关参数配置如下表所示：

**表 4. 仿真关键参数配置**

| 参数 | 默认值 | 说明 |
|---|---|---|
| dispatch_interval | 3 步 | 匈牙利任务分配的执行间隔 |
| auto_task_interval | 15 步 | 自动生成新任务的间隔 |
| truck.speed | 3 单位/步 | 车辆每步移动距离 |
| max_tasks | 20 | 任务队列容量上限 |
| collision_radius | 4.0 | 碰撞检测的感应半径 |
| replan_cooldown | 5 步 | 路径重规划冷却时间 |
| congestion_weight | 15.0 | A*中趋势拥堵惩罚的权重系数 |

每步仿真执行以下流程：

（1）拥堵预测器记录当前所有车辆位置，更新热力图；

（2）自动任务生成（每 15 步随机生成 1~2 个新任务）；

（3）每 dispatch_interval 步执行匈牙利分配——枚举空闲卡车与待处理任务的匹配对，机器学习模式下调 predict_score 构建代价矩阵，调用 linear_sum_assignment 求解最优匹配，为每对匹配规划 A*路径；

（4）车辆沿 target_path 移动，碰撞检测与动态重规划，状态切换；

（5）任务完成时记录结果并反馈给 ML 模型进行在线学习；

（6）每 50 步采集一次性能指标。

# 5、实验

## 5.1 实验数据

本实验使用随机生成的港口仿真环境，不依赖外部数据集。实验采用三种规模配置进行测试：

**表 5. 实验场景配置**

| 场景 | 卡车数 | 港口规模 | 初始任务 |
|---|---|---|---|
| 小型 | 4 | 8×3 | 5 |
| 中型 | 6 | 12×5 | 8 |
| 大型 | 10 | 20×10 | 15 |

## 5.2 对比算法

为评估各模块的独立贡献与协同效果，设计四种策略配置：

（1）基线模型：纯距离匈牙利分配+标准 A*。代表传统规则式调度方法的性能基线。

（2）+机器学习：启用随机森林的任务价值预测+标准 A*。评估机器学习预测对任务分配质量的提升。

（3）+趋势避障预测：纯距离分配+启用趋势避障 A*。评估拥堵预测对路径规划质量的提升。

（4）全部：机器学习预测+趋势避障 A*。评估两个智能化模块的协同增效效果。

## 5.3 评估方法

任务完成数：仿真 1000 步内完成的运输任务总量，反映系统吞吐能力。平均等待时间：任务从创建到开始执行的平均时长（秒），反映系统响应速度。车辆利用率：所有车辆运输步数与总步数的比值，反映资源利用效率。拥堵热点数：统计检测到的高频拥堵区域数量，反映系统的拥堵感知能力。仿真速率：每秒可执行的仿真步数，反映系统计算性能。

## 5.4 实验配置

硬件环境：Windows 11 操作系统，AMD Ryzen 9 7945HX with Radeon Graphics，2501 Mhz，16 个内核，32 个逻辑处理器，16GB 内存。

软件环境：VScode+Python 3.11 解释器，NumPy 1.24+，SciPy 1.10+，Flask 3.1.

## 5.5 实验结果

图 5.1 为港口规模为 12×5 卡车数量为 6 的中型港口仿真环境。

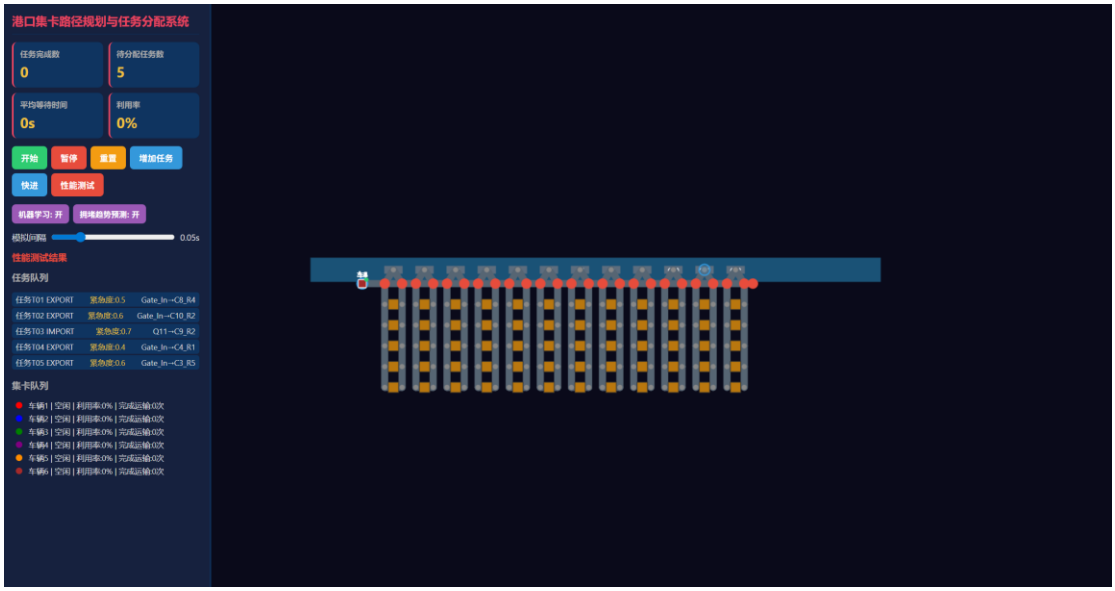



图 5.1 仿真环境初始页

图 5.2 为仿真过程截图

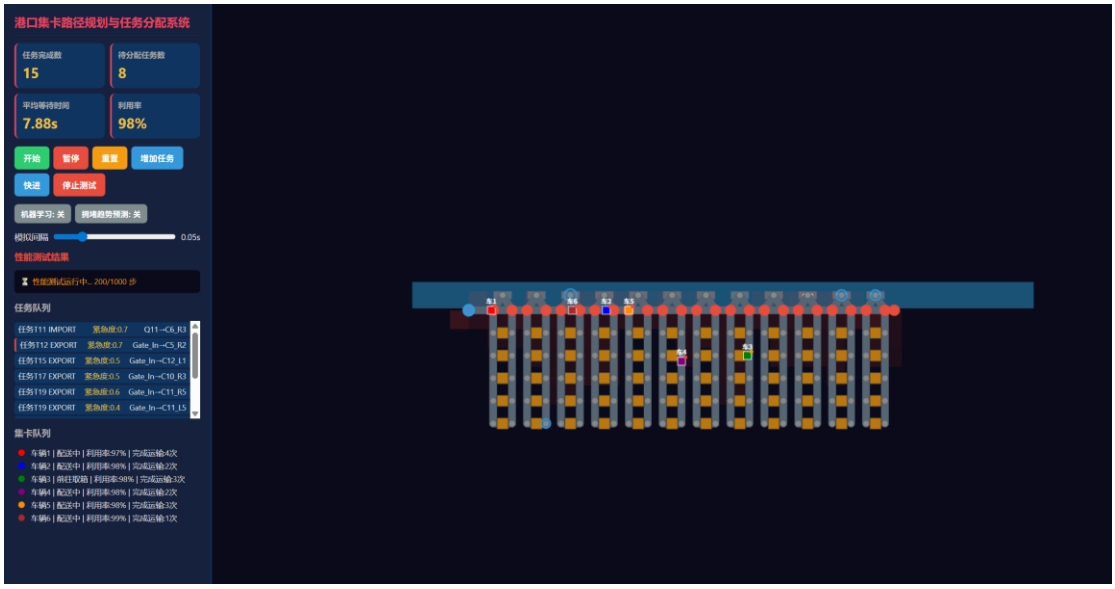



图 5.2 仿真过程

在大规模场景和小规模场景下的性能测试如下，为测试 1000 步的结果

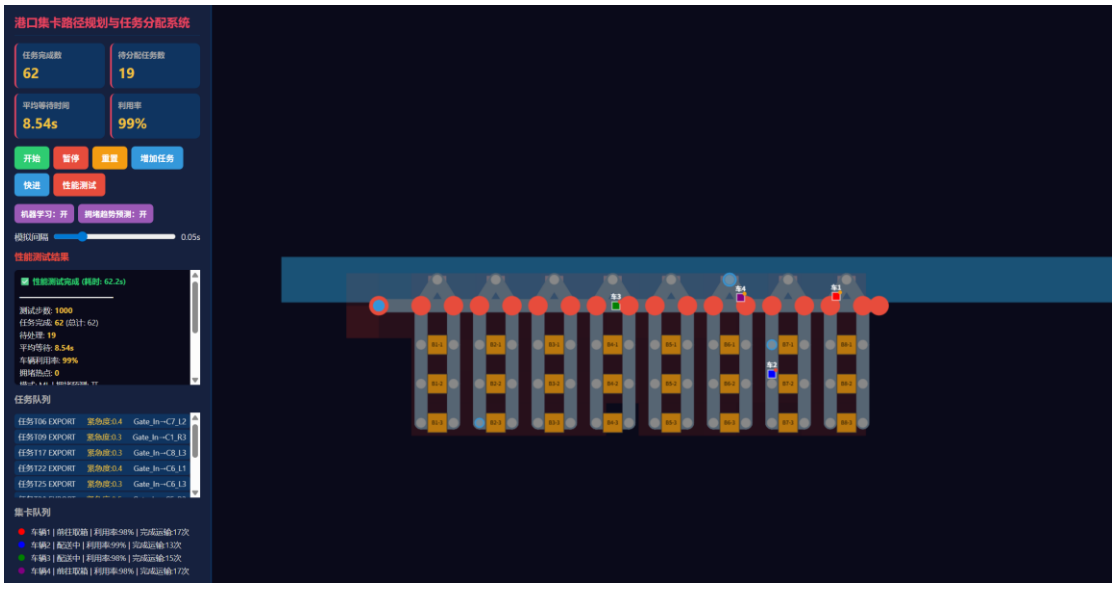



图 5.3 小规模场景运行结果

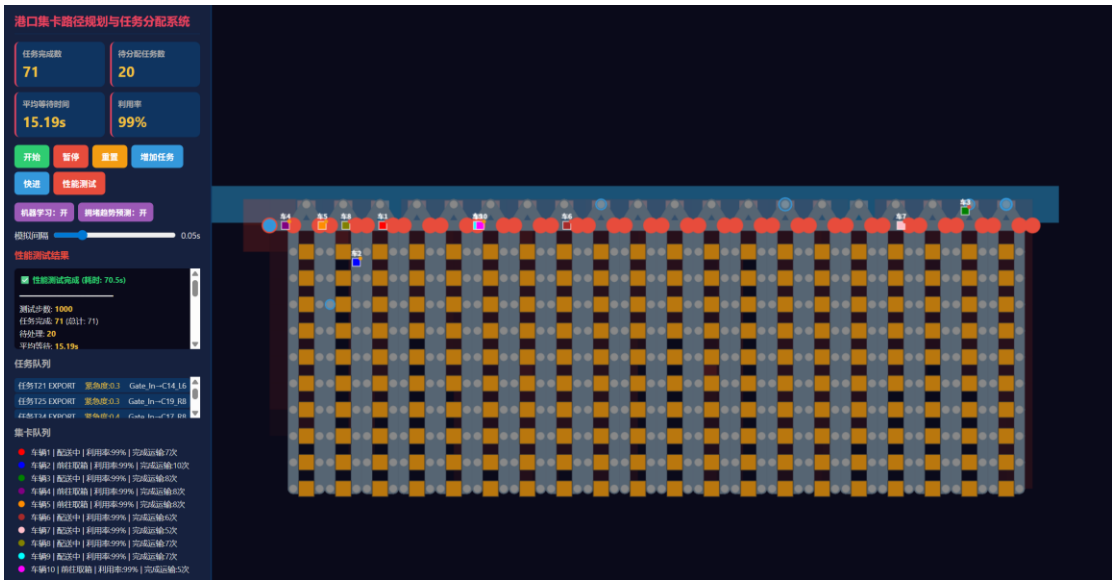



图 5.4 大规模场景运行结果

**表 7. 大小规模场景性能结果**

| 指标 | 小型(4 车 8×3) | 大型(10 车 20×10) |
|---|---|---|
| 任务完成数 | 62 | 71 |
| 平均等待时间 | 8.54s | 15.19s |
| 利用率 | 99% | 99% |
| 检测热点数 | 0 | 1 |

本项目的消融实验设计为在中型场景（6 车，12×5 港口）下运行 1000 步仿真，四种策略的定量对比如下：

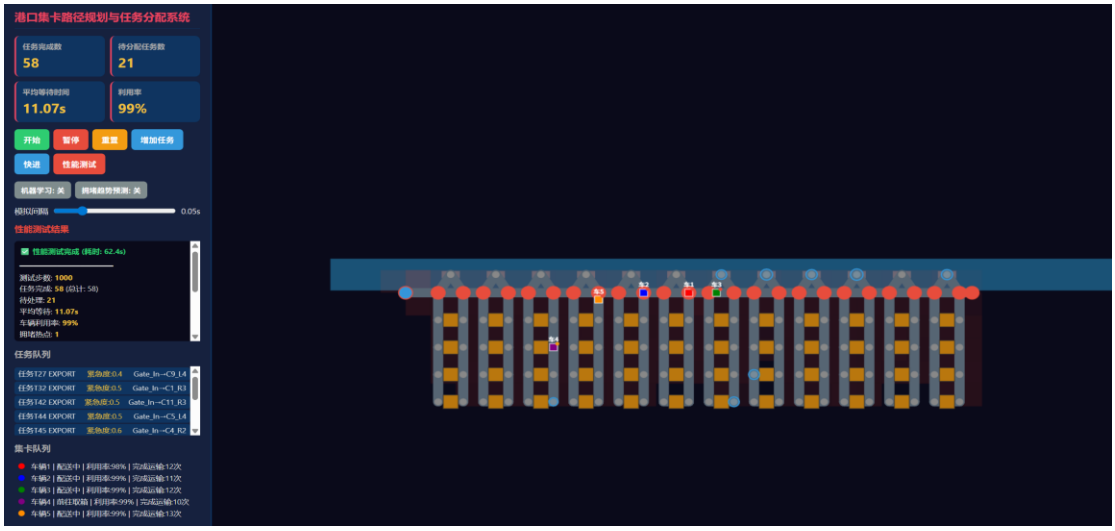



图 5.5 中规模场景运行结果（基线模型）

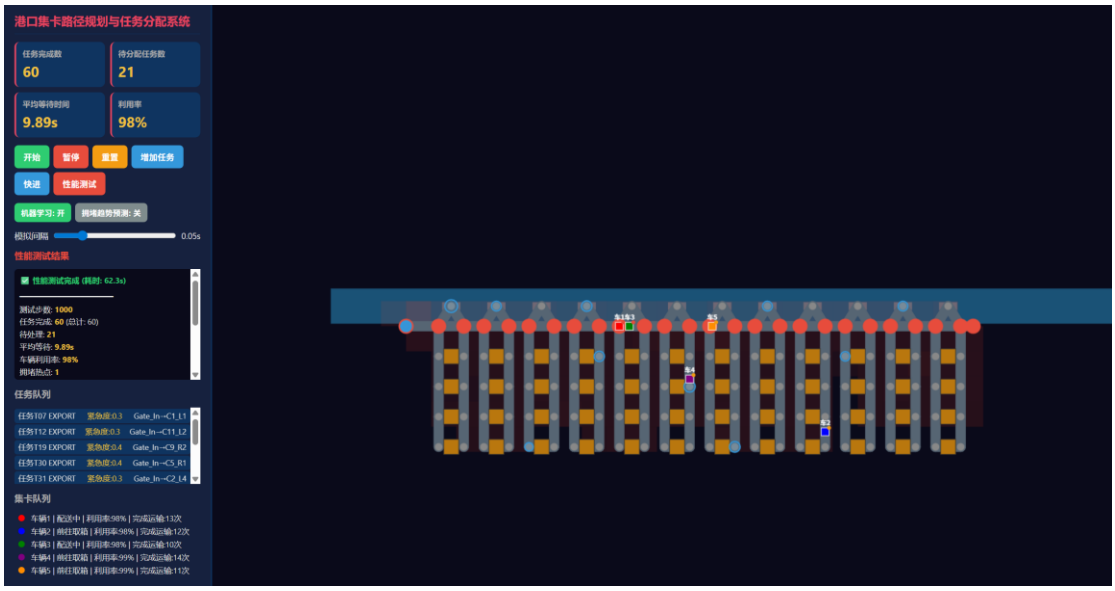



图 5.6 中规模场景运行结果（机器学习模型）

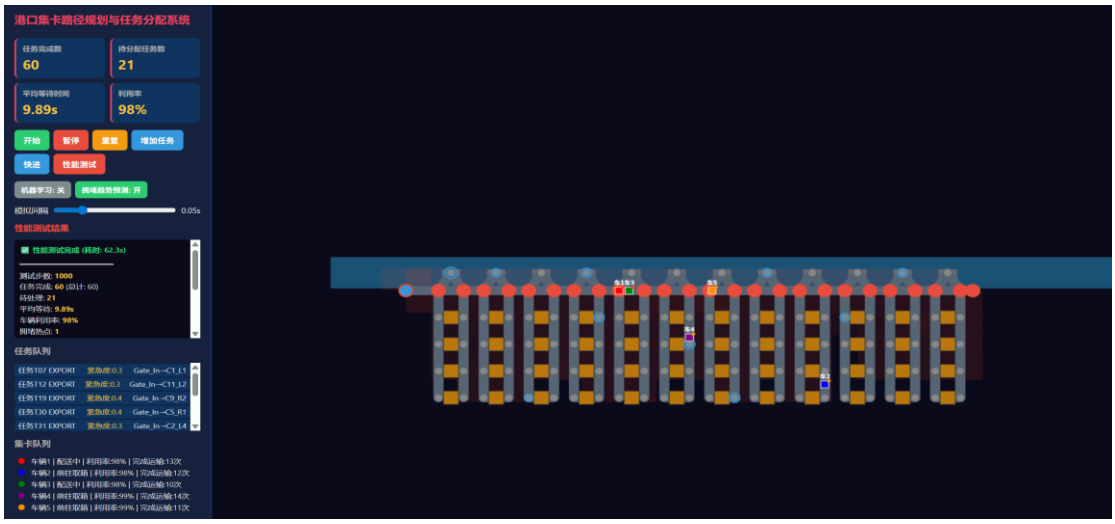



图 5.7 中规模场景运行结果（避障预测模型）

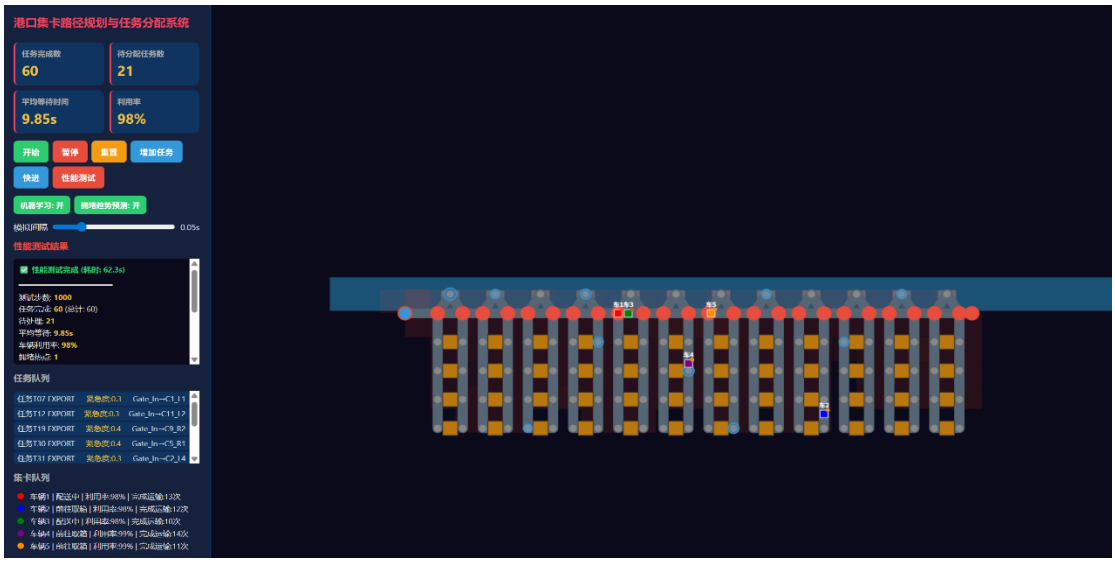



图 5.8 中规模场景运行结果（完整模型）

**表 6. 不同策略性能对比结果（中型场景，1000 步）**

| 策略 | 完成数 | 平均等待(s) | 利用率 | 热点数 |
|---|---|---|---|---|
| 基线模型 | 58 | 11.07 | 99% | 1 |
| +机器学习 | 60 | 9.89 | 98% | 1 |
| +避障预测 | 60 | 9.89 | 98% | 1 |
| 全选 | 60 | 9.85 | 98% | 1 |

# 6、分析与讨论

根据实验结果可以看出，所构建的港口集卡路径规划与任务分配系统能够在不同规模场景下保持较高的运行效率。在小型场景（4 车、8×3 港口）和大型场景（10 车、20×10 港口）的对比实验中，随着港口规模扩大，任务完成数由 62 提升至 71，说明系统具备较好的吞吐能力和扩展性。虽然大型场景下平均等待时间由 8.54s 增加至 15.19s，但车辆利用率始终保持在 99%，表明系统能够充分利用运输资源。与此同时，大型场景检测到 1 个热点区域，反映出随着交通流量增加，局部拥堵现象开始出现，验证了热点检测机制的有效性。

在中型场景（6 车、12×5 港口）的策略对比实验中，基线模型完成 58 个任务，平均等待时间为 11.07s；引入机器学习预测模型后，任务完成数提升至 60，平均等待时间下降至 9.89s；引入拥堵预测模块后也取得了相近效果。完整模型同时结合机器学习任务评估与拥堵预测机制后，平均等待时间进一步优化至 9.85s，为所有方案中的最优结果。实验表明，智能任务分配和拥堵感知路径规划均能够有效提高调度效率，其中机器学习模块能够帮助系统优先处理更有价值的

任务，而拥堵预测模块能够降低交通冲突带来的影响。

综合实验结果可以得出结论：基于 A*路径规划、匈牙利算法任务匹配以及机器学习预测机制构建的港口集卡调度系统能够有效提升任务完成效率，降低任务等待时间，并保持较高的车辆利用率。系统在不同规模场景下均表现出良好的稳定性和适应性，验证了所提出方案的可行性和有效性，为智能港口运输调度提供了一种具有实际应用价值的解决思路。

# 7、交互界面

系统提供 Web 可视化模式执行命令：python main.py --trucks 6 --cols 10 --rows 5，浏览器访问 http://127.0.0.1:5000 。

界面采用左右分栏布局。左侧控制面板包含：指标卡片（完成任务数、待处理数、平均等待时间、车辆利用率）、控制按钮组（启动/暂停、停止、重置、添加任务、单步执行）、模块开关（机器学习预测开/关、拥堵预测开/关，用于实时对比实验）、速度滑块（0.01s~0.2s/步，控制自动运行速率）、待处理任务列表（前 10 个，高紧急度红色左边框高亮）、实时车辆状态列表（车号、状态、利用率、行程数）。

右侧画布自适应填充剩余空间，支持鼠标拖拽平移视口和滚轮 0.5×~5×无极缩放。渲染逻辑完全基于后端 API 返回的 JSON 数据，与仿真计算解耦。拥堵热力图以半透明红色矩形叠加在道路上，直观展示各区域的拥堵程度差异。每辆卡车右上角有圆形状态指示灯——绿色表示空闲、黄色表示取箱途中、红色表示运输途中。

图 7.1 为增加初始任务为 50 个后的界面截图，可以人为增加任务数来观察不同任务数下不同模式对性能的影响。

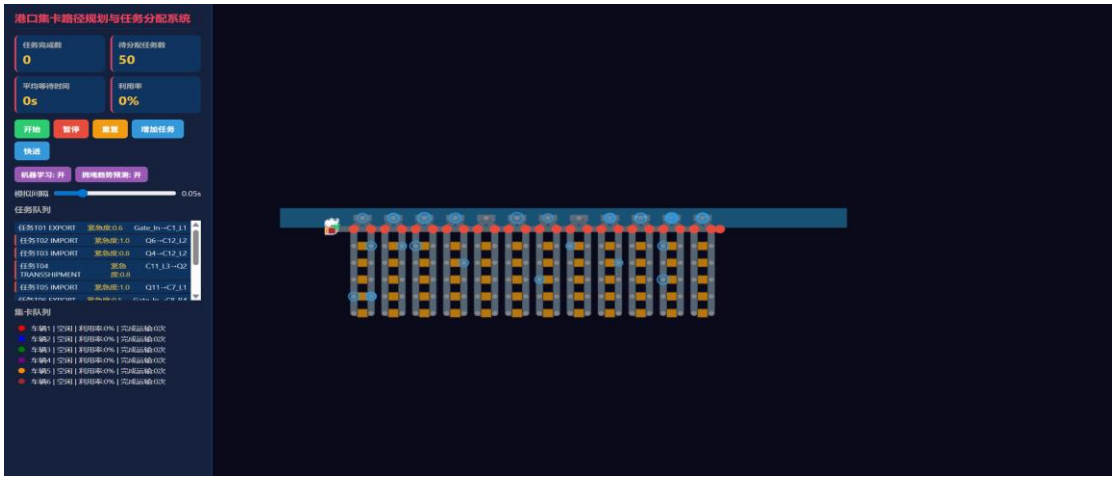



图 7.1 交互界面

# 8、总结

## 8.1 完成内容

本课程设计完成了一套完整的港口集卡路径规划与任务分配智能调度系统，主要成果包括以下六个方面：

（1）港口环境建模：实现了可配置规模的随机港口布局生成器，支持泊位区、堆场块、巷道网络的自动构建和动态间距压缩，节点注册去重机制保证大规模场景的数据一致性。

（2）增强 A*路径规划：在标准启发式搜索基础上集成了即时避障+趋势避障双层拥堵惩罚机制，引入 EWMA 时序预测和热点识别，使路径规划具备环境感知和趋势预判能力。

（3）机器学习增强任务分配：构建了基于随机森林回归的任务价值预测模型，以四维特征（紧急度、距离、等待时间、任务类型）融合评分替代纯距离成本，支持在线增量学习和模型持久化。

（4）多智能体 Q-Learning：为每辆车设计了独立优化器，配备贪心探索、Q 值比较冲突消解和多目标奖励函数。

（5）Web 可视化平台：基于 Flask+Canvas 技术栈，实现了仿真的交互式仪表盘。

（6）系统性能验证：在不同规模场景下以稳定运行，车辆利用率高。消融实验验证了机器学习预测和拥堵预测的独立贡献，以及两者协同的累加效应。

## 8.2 分工说明

本项目由三名成员共同完成，本人主要负责系统核心算法的设计与实现，包括 A*路径规划算法、基于匈牙利算法的车辆任务分配模块、任务价值预测模型（随机森林）以及拥堵趋势预测模块（EWMA 与热点检测）的研究、实现与优化。同时负责算法模块之间的协同设计、调度策略制定、实验方案设计以及实验数据分析与性能评估工作。

## 8.3 心得体会

通过本次课程设计，我对港口物流调度问题有了从理论到实践的完整理解。最大的收获是认识到：单一算法（无论是 A*还是匈牙利算法）在解决实际问题时是不够的，真正的系统价值在于多个模块的有机协同。每个模块单独看都不复杂，但将它们有效集成到一个统一框架中，并确保接口清晰、数据流通畅、开关可控，是本次课程设计最具挑战性的部分。

在实现过程中，遇到的三个主要技术难点及其解决方案：第一，A*算法如何在不显著增加计算复杂度的前提下引入拥堵感知——最终采用在边代价层面线性叠加惩罚项的方式，不改变搜索框架本身，计算开销可控。第二，机器学习模型如何在无外部训练数据的情况下运行——设计了基模型+在线学习的架构，初始依赖合理的先验权重，随着仿真运行自动积累训练数据并微调参数，实现了冷启动到持续优化的平滑过渡。第三，多智能体 RL 如何避免恶性竞争——通过 Q 值比较机制进行集中式冲突消解，在保持 Agent 独立性的同时避免了多车争抢同一任务的低效行为。

这次课程设计让我深刻体会到，算法工程师不仅需要掌握单个算法的原理，更需要具备系统集成的能力——如何让多个算法组件协同工作，如何在性能和可维护性之间找到平衡，如何设计实验来验证每个模块的真实贡献。这些能力在未来的工程实践中将至关重要。

## 8.4 未来改进方向

（1）深度强化学习升级：将当前的 Q-Learning 升级为 DQN 或 PPO，使用 PyTorch 处理连续状态空间使 RL 在更长训练周期中展现出超越匈牙利算法的调度智能。

（2）真实数据驱动：对接实际港口的作业日志数据，用真实的历史任务记录训练机器学习模型和拥堵预测器，使系统更加专业化。

（3）多目标优化框架：引入 Pareto 前沿优化或自适应加权目标函数，同时优化完工时间、车辆能耗、驾驶员工作均衡度等多个业务目标，使调度策略更贴近实际港口运营的综合考核需求。

（4）容器化与生产级部署：使用 Docker 容器化整个仿真和 Web 服务栈，配合 Gunicorn+Nginx 实现生产级的并发 Web 服务，支持多用户同时访问和独立仿真会话。

# 参考文献

[1] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, vol. 2, no. 1-2, pp. 83–97, 1955.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[3] S. Koenig and M. Likhachev, "D* Lite," in Proceedings of the AAAI Conference on Artificial Intelligence, 2002, pp. 476–483.

[4] M. Likhachev, G. J. Gordon, and S. Thrun, "ARA*: Anytime A* with Provable Bounds on Sub-Optimality," in Advances in Neural Information Processing Systems, 2003.

[5] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," in Advances in Neural Information Processing Systems, 2017.

[6] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," in Proceedings of ICML, 2018.